

SISTEMAS INTELIGENTES
ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Práctica 1:

**"RESOLUCIÓN DE PROBLEMAS MEDIANTE BÚSQUEDA
EN UN ESPACIO DE ESTADOS"**

DEPARTAMENTO DE SISTEMAS INFORMÁTICOS Y COMPUTACIÓN
UNIVERSITAT POLITÈCNICA DE VALÈNCIA

Valencia, Septiembre 2026

1. Introducción

El objetivo de la práctica consiste en evaluar la eficiencia, coste temporal y espacial de distintas estrategias de búsqueda aplicadas al juego del n-puzzle. Para ello se proporciona una librería "search.py" donde se implementan diversos algoritmos de los vistos en clase, así como una serie de métodos de ayuda para la ejecución y evaluación de las soluciones propuestas. También se incluyen diversos notebooks donde se plantean diversos ejercicios.

2. Aspectos generales del juego del puzzle

En este apartado se resumen brevemente los aspectos más importantes del juego del puzzle.

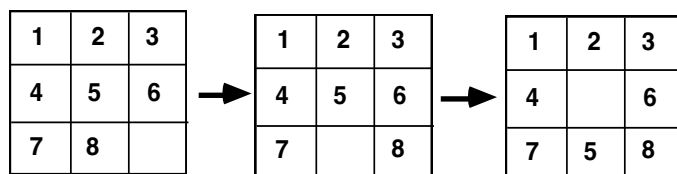
2.1. Representación

El juego del puzzle se representa sobre un tablero de $n \times n$ casillas.

Si particularizamos a un ejemplo completo, supongamos un juego donde $n=3$, y por tanto el número de casillas es 9. 8 de las casillas contienen una pieza o ficha que se puede deslizar a lo largo del tablero horizontal y verticalmente. Las fichas vienen marcadas con los números del 1 al 8. Hay una casilla libre en el tablero que permite los movimientos de las fichas.

El número total de estados o configuraciones del problema que se puede generar en el juego del puzzle es $9! = 362.880$ estados.

Ejemplos de movimientos:



El **objetivo** del problema es obtener, como solución, la secuencia de 'movimientos del cuadro vacío' que transforma el estado inicial al estado objetivo:

1	2	3
8		4
7	6	5

2.2. Operadores

Los operadores del juego del puzzle se establecen como los movimientos del cuadro vacío. Estos movimientos son: arriba, abajo, derecha, izquierda. Nótese que, sólo cuando el cuadro vacío está en la casilla central, se podrán realizar los 4 movimientos. Si el cuadro vacío está en una esquina entonces el número de movimientos válidos es 2; en cualquier otro caso existen 3 posibles movimientos para el cuadro vacío.

3. Estrategias de búsqueda.

Las estrategias de búsqueda que ofrece el programa *puzzle* son:

BFS. Esta es la estrategia de **Anchura** que utiliza $f(n)=g(n)$ para la expansión de nodos, donde $g(n)$ es el factor coste asociado al nivel de profundidad del nodo n en el árbol de búsqueda.

DFS (Backtracking). Esta estrategia implementa una búsqueda en **Profundidad** con backtracking o **Profundidad TREE-SEARCH**. Para ello, utiliza la función $f(n)=g(n)$, donde $g(n)$ es el factor coste asociado al nivel de profundidad del nodo n en el árbol de búsqueda. A diferencia de DFS (Graph Search), esta estrategia solo mantiene en memoria los nodos explorados que pertenecen al camino actual (lista PATH), el resto de nodos explorados no se mantienen en la lista CLOSED sino que se eliminan.

Voraz (Manhattan). Esta es una estrategia que implementa un *Algoritmo voraz* siguiendo la función $f(n)=D(n)$, donde $D(n)$ es la distancia de Manhattan.

ID. Esta estrategia implementa una búsqueda por **Profundización Iterativa** (Iterative Deepening). Se trata de realizar sucesivas búsquedas en profundidad con backtracking hasta que se alcanza la solución. Cada nueva búsqueda incrementa el nivel de profundidad de la exploración en 1.

A* Manhattan. Esta es una búsqueda de tipo A que utiliza la distancia de Manhattan como heurística; la expansión de los nodos se realiza siguiendo la función $f(n)=g(n)+D(n)$, donde $g(n)$ es el factor coste correspondiente a la profundidad del nodo n en el árbol de búsqueda y $h(n)=D(n)$ es la distancia de Manhattan, es decir, la suma de las distancias en horizontal y vertical de cada ficha del puzzle desde su posición actual a la posición que ocupará en el objetivo.

A* (MD + LC). Esta estrategia implementa una búsqueda de tipo A que utiliza una heurística informada basada en la combinación de la distancia de Manhattan y los conflictos lineales (LC). La expansión de nodos se realiza con la función $f(n)=g(n)+h(n)$, donde $g(n)$ es el coste asociado al nivel de profundidad del nodo n y $h(n)=D(n)+LC(n)$, siendo $D(n)$ la distancia de Manhattan y $LC(n)$ el número de conflictos entre fichas mal colocadas en la misma fila o columna que deben cruzarse. Esta heurística mejora la precisión respecto a A* Manhattan, ya que añade penalizaciones por conflictos que implican movimientos adicionales necesarios.

IDA* Manhattan. Esta estrategia de búsqueda implementa una búsqueda IDA* donde el límite de la búsqueda viene determinado por el valor- f , es decir, por la función $f(n)=g(n)+D(n)$, donde $g(n)$ es el factor de coste y $h(n)=D(n)$ es la distancia de Manhattan. El límite de una iteración i del

algoritmo IDA* es el valor-f más pequeño de cualquier nodo que haya excedido el límite en la iteración anterior $i-1$.

NOTA IMPORTANTE:

La estrategia DFS (Backtracking) funciona con un nivel máximo de profundidad. Si el nivel máximo actual es menor que el nivel de la solución óptima entonces la estrategia no encontrará solución.

4. Descripción del código proporcionado.

Se proporcionan **4 ficheros** que conforman el entorno de trabajo necesario para realizar la práctica:

1. **search.py**

Contiene la implementación de diversos algoritmos de búsqueda clásicos (como BFS, DFS, A*, IDA*) y heurísticas (como Manhattan). También incluye funciones auxiliares para visualizar estados, mostrar caminos de solución, y comparar resultados entre algoritmos.

2. **puzzle-p1.ipynb**

Notebook de introducción a los algoritmos de búsqueda aplicados al puzzle. Sirve para familiarizarse con el uso del fichero search.py, la ejecución de estrategias de búsqueda, y la visualización de soluciones.

3. **puzzle-p2.ipynb**

Notebook centrado en cómo implementar y probar heurísticas nuevas, así como en la comparación de heurísticas. Es el cuaderno principal para desarrollar y validar las nuevas funciones heurísticas solicitadas en la práctica.

4. **alg-compare-p3.ipynb**

Notebook avanzado orientado a realizar comparaciones detalladas entre heurísticas. Incluye visualización gráfica y análisis cuantitativo para apoyar la justificación de resultados.

4.1 Descripción en detalle de “search.py”

En el fichero search.py contiene la implementación de varios algoritmos de búsqueda clásicos junto con métodos auxiliares. **Este fichero no debe modificarse.**

A continuación, se describen los métodos que pueden ser usados durante la práctica:

Funciones de heurísticas proporcionadas

`getManhattanDistance(state, end_state)`

Calcula la **distancia de Manhattan total** entre el estado actual y el estado objetivo. Es una heurística admisible para problemas tipo n-puzzle.

- **Parámetros:**

- `state`: lista que representa el estado actual del tablero (por ejemplo, [1, 2, 3, 4, 5, 6, 7, 8, 0]).
- `end_state`: lista con el estado objetivo.

- **Retorna:**

Un entero que representa la suma de las distancias Manhattan de todas las piezas (excepto el 0/blanco) entre su posición actual y la deseada.

`getLinear_conflict(state, end_state)`

Calcula la **cantidad de conflictos lineales** en filas y columnas entre `state` y `end_state`. Un conflicto lineal ocurre cuando dos piezas están en la misma fila o columna que su destino, pero en orden incorrecto. Cada conflicto seleccionado aporta 2, pero una ficha no puede formar parte de más de un conflicto dentro de la misma fila o columna.

- **Parámetros:**

- `state`: lista que representa el estado actual del tablero.
- `end_state`: lista que representa el estado objetivo.

- **Retorna:**

Un entero que representa el número total de conflictos lineales detectados multiplicado por 2.

`get_md_plus_linear_conflict(state, end_state)`

Combina dos heurísticas: **distancia de Manhattan** y **conflictos lineales**. Es una heurística más informada que solo Manhattan y sigue siendo admisible.

- **Parámetros:**

- `state`: lista con el estado actual.
- `end_state`: lista con el estado objetivo.

- **Retorna:**

Un valor entero: `getManhattanDistance + getLinear_conflict`.

Funciones para interpretación y visualización

`check_consistency(heuristic, initial_state, end_state, size, max_states=10000)`

Realiza una comprobación empírica de la consistencia de una función heurística. A partir de un estado inicial, recorre estados alcanzables y comprueba, para cada transición entre un estado n y uno de sus sucesores n' , que se cumple:

$$h(n) \leq 1 + h(n')$$

El valor 1 corresponde al coste de realizar un movimiento en el n -puzzle.

- **Parámetros:**

- **heuristic:** función heurística que se desea comprobar. Debe recibir los argumentos (state, end_state).
- **initial_state:** lista que representa el estado desde el que comienza la exploración.
- **end_state:** lista que representa el estado objetivo.
- **size:** dimensión del tablero, por ejemplo, 3 para un puzzle 3×3.
- **max_states:** número máximo de estados que se comprobarán. Su valor predeterminado es 10000. Puede utilizarse None para explorar todos los estados alcanzables.

- **Retorna:**

Un diccionario con los siguientes campos:

- **consistent:** indica el resultado de la comprobación:
 - False si se ha encontrado un contraejemplo.
 - None si no se ha encontrado ningún contraejemplo, pero se alcanzó el límite max_states.
 - True si se exploraron todos los estados alcanzables sin encontrar contraejemplos.
- **complete:** indica si se completó la exploración de todos los estados alcanzables.
- **states_checked:** número de estados examinados.
- **counterexample:** contiene la transición que incumple la condición de consistencia, o None si no se ha encontrado ninguna.

Cuando se encuentra un contraejemplo, este campo incluye:

- **state:** estado n .
- **child:** estado sucesor n' .
- **h_state:** valor $h(n)$.
- **h_child:** valor $h(n')$.
- **step_cost:** coste de la transición, que en este problema es 1.

Ejemplo de uso:

```
result = check_consistency(
    nueva_heuristica,
    initial_state,
    end_state,
    size,
    max_states=10000
)

print(result)
```

Si se encuentra un incumplimiento, el resultado tendrá una estructura similar a:

```
{
    'consistent': False,
    'complete': False,
```

Práctica 1: Búsqueda en un espacio de estados

```
'states_checked': 25,
'counterexample': {
    'state': [1, 2, 3, 4, 5, 6, 0, 7, 8],
    'child': [1, 2, 3, 4, 5, 6, 7, 0, 8],
    'h_state': 4,
    'h_child': 2,
    'step_cost': 1
}
```

En este caso se incumple la condición $4 \leq 1 + 2$, por lo que la heurística no es consistente.

`visualize_state(state)`

Muestra en consola el estado del tablero con formato de matriz, para facilitar la lectura humana.

- **Parámetros:**
 - `state`: lista de enteros representando el estado del puzzle.
- **Salida:** Imprime el tablero en consola.

`show_path_for_algorithm(results, algorithm_name, size)`

Busca en un diccionario de resultados la información de un algoritmo en concreto y muestra su camino de solución.

- **Parámetros:**
 - `results`: diccionario que asocia nombre del algoritmo con sus resultados.
 - `algorithm_name`: nombre del algoritmo cuyo resultado se quiere mostrar.
 - `size`: dimensión del tablero.
- **Salida:** Imprime en consola los pasos de la solución correspondiente.

`plot_algorithm_comparison(results)`

Grafica una comparación de algoritmos basada en tiempo de ejecución, nodos generados, coste, etc.

- **Parámetros:**
 - `results`: diccionario con resultados para varios algoritmos (ver estructura estándar del diccionario `results` en `run_algorithm`).
- **Salida:**
Gráfica generada mediante `matplotlib`.

`show_results(results)`

Imprime por consola un resumen de los resultados obtenidos por cada algoritmo (tiempo, profundidad, nodos, etc.).

- **Parámetros:**

Práctica 1: Búsqueda en un espacio de estados

- o `results`: diccionario con los resultados de ejecución para varios algoritmos (ver estructura estándar del diccionario `results` en `run_algorithm`).
- **Salida:**
Tabla textual con los datos comparativos.

`show_path_puzzle(results, algorithm_name, size, width=420)`

Muestra de forma interactiva el camino de solución de un algoritmo en un notebook, mediante un tablero generado con HTML, CSS y JavaScript. Permite avanzar y retroceder paso a paso, ir al primer o al último estado, reproducir o pausar la secuencia y ajustar la velocidad. En cada paso se resalta la ficha movida y se indica el progreso de la solución.

- **Parámetros:**
 - o `results`: diccionario con los resultados de ejecución; debe contener el camino de solución en `results[algorithm_name]['path']`.
 - o `algorithm_name`: nombre del algoritmo cuyo camino se quiere visualizar; debe coincidir con una clave de `results`.
 - o `size`: dimensión del tablero (por ejemplo, 3 para un puzzle de 3x3).
 - o `width`: anchura máxima de la visualización en píxeles (opcional; 420 por defecto).
- **Salida:**
Visualización interactiva dentro del notebook. Ejemplo de visualización:



Generación de estados del puzzle

`generate_random_state(size, end_state)`

Genera un **estado aleatorio válido (resoluble)** del puzzle, a partir del estado final objetivo. La función aplica movimientos aleatorios al estado objetivo sin repetir estados, garantizando que el resultado es alcanzable.

- **Parámetros:**
 - o `size`: dimensión del tablero (por ejemplo, 3 para un puzzle de 8 piezas).
 - o `end_state`: lista que representa el estado objetivo (por ejemplo, ordenado del 1 al $n^2 - 1$ y el 0 al final).

- **Retorna:**
Lista de enteros que representa un nuevo estado del puzzle, aleatorio pero resoluble.

- **Ejemplo de uso:**

```
goal = generate_ordered_state(3)
initial = generate_random_state(3, goal)
```

generate_ordered_state(n)

Crea el **estado ordenado estándar** del puzzle, donde las piezas están dispuestas de menor a mayor, con el espacio en blanco (0) en la última posición.

- **Parámetros:**
 - n: dimensión del tablero (por ejemplo, 3 para un 3x3).
- **Retorna:**
Lista de enteros de tamaño $n * n$, ordenada (por ejemplo, [1, 2, 3, 4, 5, 6, 7, 8, 0] para $n=3$).

generate_spiral_state(n)

Genera un estado objetivo **con ordenación en espiral (modo caracol)**, útil como alternativa al orden clásico. Las piezas se colocan en espiral desde la esquina superior izquierda hasta el centro.

- **Parámetros:**
 - n: dimensión del tablero ($n \times n$).
- **Retorna:**
Lista de enteros de tamaño $n * n$, colocados en orden espiral, terminando con el 0 en el centro (si n es impar).
- **Ejemplo de retorno para $n=3$:** (en realidad retorna una lista)

```
[1, 2, 3,
 8, 0, 4,
 7, 6, 5]
```

Por otro lado, search.py incluye un registro común de algoritmos y heurísticas.

Métodos auxiliares para ejecutar algoritmos

```
run_algorithm(algorithm_name, initial_state, end_state, size, depth=50,
             heuristic_func=None)
```

Este método ejecuta un único algoritmo de búsqueda especificado por su nombre, sobre un estado inicial y un estado objetivo del puzzle. Devuelve un diccionario con la información de la solución encontrada (o vacío si no hay solución).

- **Parámetros:**
 - algorithm_name: cadena de texto con el nombre del algoritmo a ejecutar (ver lista soportada).
 - initial_state: lista que representa el estado inicial del puzzle.

- `end_state`: lista que representa el estado objetivo.
- `size`: tamaño del tablero (por ejemplo, 3 para puzzles 3x3).
- `depth`: profundidad máxima (solo se usa en algunos algoritmos, como DFS-B con límite).
- `heuristic_func`: función heurística opcional (**no se usa en esta implementación**, las heurísticas están integradas según el nombre del algoritmo).
- **Retorna:**
Diccionario con una única clave (el nombre del algoritmo) y como valor otro diccionario con:
 - `'path'`: lista de estados desde el inicial al objetivo.
 - `'cost'`: coste total de la solución.
 - `'depth'`: profundidad alcanzada.
 - `'nodes_generated', 'nodes_expanded', 'time', 'max_nodes'`: métricas de rendimiento.
- **Algoritmos soportados:**
 - `'BFS'`: Búsqueda en anchura (costo uniforme sin heurística).
 - `'DFS-B'`: Búsqueda en profundidad limitada (versión propia con control recursivo).
 - `'Voraz (Manhattan)'`: Búsqueda voraz usando la distancia Manhattan como heurística.
 - `'ID'`: Búsqueda iterativa en profundidad (IDDFS).
 - `'A* (Manhattan)'`: Algoritmo A* usando la heurística de Manhattan.
 - `'A (MD + LC)'`: A* con heurística mejorada que suma Manhattan y conflictos lineales.
 - `'IDA* (Manhattan)'`: Búsqueda A* iterativa en profundidad con heurística de Manhattan.

`run_all_algorithms(initial_state, end_state, size, depth=50)`

Este método ejecuta **automáticamente todos los algoritmos soportados** sobre el mismo estado inicial y objetivo. Se utiliza para comparar sus resultados.

- **Parámetros:**
 - `initial_state`: lista que representa el estado inicial del puzzle.
 - `end_state`: lista que representa el estado objetivo.
 - `size`: dimensión del tablero.
 - `depth`: profundidad máxima (usada en algoritmos con límite como DFS-B).

- **Retorna:**
Diccionario que asocia el nombre de cada algoritmo con su correspondiente resultado (estructura igual a la devuelta por `run_algorithm`).
- **Algoritmos ejecutados:**
 - 'BFS'
 - 'DFS-B'
 - 'Voraz (Manhattan)'
 - 'ID'
 - 'A* (Manhattan)'
 - 'A (MD + LC)'
 - 'IDA* (Manhattan)'

```
register_heuristic(name, heuristic, strategy='astar', include_in_all=True,  
replace=False)
```

Registra una nueva heurística para que pueda utilizarse directamente mediante `run_algorithm` y, por defecto, también mediante `run_all_algorithms`.

- **Parámetros:**
 - `name`: etiqueta con la que se identificará la heurística.
 - `heuristic`: función que recibe (`state`, `end_state`) y devuelve su valor heurístico.
 - `strategy`: estrategia que utilizará la heurística: 'astar', 'greedy' o 'ida'.
 - `include_in_all`: si es `True`, la heurística se incluye automáticamente en `run_all_algorithms`.
 - `replace`: permite reemplazar un registro existente con el mismo nombre cuando vale `True`.

Representación de los estados y ejecución de los algoritmos

Los estados del puzzle se representan como matrices $n \times n$, que se convierten en listas unidimensionales para ser procesadas por los algoritmos. A continuación, se muestra un ejemplo de código para un puzzle de 3x3:

```
size = 3 # Puzzle 3x3  
  
# Estado inicial y objetivo como matrices  
initial_state_matrix = [  
    [7, 8, 1],  
    [4, 0, 6],  
    [2, 3, 5]  
]  
  
end_state_matrix = [  
    [1, 2, 3],  
    [8, 0, 4],  
    [7, 6, 5]  
]
```

Práctica 1: Búsqueda en un espacio de estados

```
# Conversión a listas unidimensionales
initial_state = [num for row in initial_state_matrix for num in row]
end_state = [num for row in end_state_matrix for num in row]
```

Este código define el tamaño del puzzle como 3x3 (`size = 3`) y representa tanto el estado inicial como el estado objetivo en forma de matrices. A continuación, convierte estas matrices en listas unidimensionales (`initial_state` y `end_state`), que es el formato requerido por los algoritmos de búsqueda para procesar los estados del puzzle.

A continuación, se comprueba si el puzzle es resoluble y se ejecutan todos los algoritmos:

```
results = {}
if isSolvable(initial_state, end_state):
    results = run_all_algorithms(initial_state, end_state, size, depth=40)
    show_results(results)
    plot_algorithm_comparison(results)
else:
    print("The puzzle is not solvable.")
```

Este fragmento de código comprueba si el estado inicial del puzzle puede transformarse en el estado objetivo usando la función `isSolvable`. Si lo es, ejecuta todos los algoritmos de búsqueda mediante `run_all_algorithms` y guarda los resultados en el diccionario `results`. Luego, muestra un resumen de las métricas con `show_results` y una comparación gráfica con `plot_algorithm_comparison`.

5. ¿Cómo añadir una nueva heurística?

Supongamos que se quiere incluir una función que implemente una nueva función heurística. A modo de ejemplo explicamos la función `getManhattanDistance` ya incluida en el código:

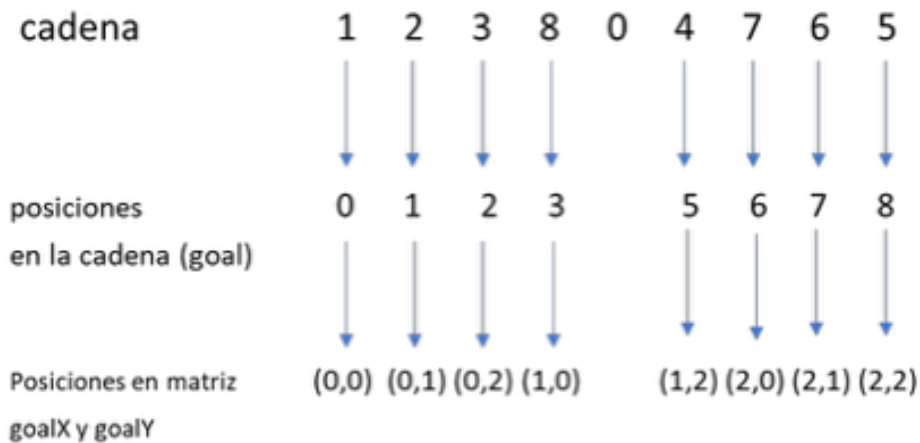
```
def getManhattanDistance(state, end_state): #state es el estado actual a evaluar,
end_state es el estado objetivo
    tot = 0
    size=int(math.sqrt(len(state)))
    for i in range(1, len(state)): #recorre las n piezas
        goal = end_state.index(i) #posición de la pieza i en el estado final (end_state)
        goalX = goal // size      #Pasamos el end state de cadena a una matriz de nxn
        goalY = goal % size       #fila de la pieza (goalX) y columna (goalY)
        idx = state.index(i)      #misma operación pero para la fila (idx) y columna (idy)
        itemX = idx // size        #que ocupa esa pieza en el estado actual
        itemY = idx % size
        tot += (abs(goalX - itemX) + abs(goalY - itemY)) #cálculo de distancias
    return tot
```

La función recibe en `state` el estado actual a evaluar. Para cada una de las 8 piezas (el espacio en blanco se representa con un 0 pero no es una pieza) se calcula la distancia desde la posición actual de esa pieza a su posición en el estado final (representado en la variable `end_state`). Las

Práctica 1: Búsqueda en un espacio de estados

operaciones intermedias son para para calcular las posiciones de cada pieza en una matriz de nxn como en el puzzle a partir de la representación lineal en una cadena.

Por ejemplo, para el estado final (`end_state`), la transformación de representación lineal (cadena) a matriz en un puzzle de 3x3 sería:



Una vez implementada la nueva función heurística, se registra mediante `register_heuristic`. No es necesario modificar `run_algorithm` ni `run_all_algorithms`.

```
def nueva_heuristica(state, end_state):  
    # Cálculo del valor heurístico  
    return valor
```

```
register_heuristic('Nueva Heurística', nueva_heuristica)
```

El primer argumento es la etiqueta utilizada para ejecutar y mostrar la heurística. El segundo es la función que se acaba de implementar. Como no se indica ninguna estrategia, se utiliza A* por defecto.

Si se desea utilizar otra estrategia de búsqueda, puede indicarse expresamente:

```
register_heuristic('Nueva A*', nueva_heuristica, strategy='astar')  
register_heuristic('Nueva Voraz', nueva_heuristica, strategy='greedy')  
register_heuristic('Nueva IDA*', nueva_heuristica, strategy='ida')
```

Por defecto, toda heurística registrada se incorpora automáticamente a `run_all_algorithms`. Si solo se quiere ejecutar por separado, se utiliza:

```
register_heuristic(  
    'Nueva Heurística',  
    nueva_heuristica,  
    include_in_all=False,  
)
```

Para ejecutar por separado la nueva heurística se puede ejecutar lo siguiente:

```
# suponiendo que se han inicializado initial_state, end_state y size
result = run_algorithm('Nueva Heurística', initial_state, end_state, size)
show_results(result)
```

También se puede ejecutar la nueva heurística junto con el resto de los algoritmos:

```
# suponiendo que se han inicializado initial_state, end_state, size y max_prof
results = run_all_algorithms(initial_state, end_state, size, max_prof)
show_results(results)
```

TRABAJO A REALIZAR

En este apartado se expone la tarea que tiene que realizar cada grupo de prácticas (bien individual o un grupo de dos personas como máximo) sobre el n-puzzle.

Se pide implementar diversas funciones heurísticas y justificar su comportamiento. El día del examen de prácticas todos los estudiantes deben subir a su tarea:

- El código Python de cada una de las heurísticas (ficheros de notebook) incluyendo las respuestas y justificaciones a las cuestiones formuladas

En el caso de un grupo de prácticas de dos personas, ambas subirán los mismos ficheros a sus correspondientes tareas. **NOTA: Indicad claramente los nombres de las dos personas en el fichero de texto que contiene las respuestas a las preguntas formuladas.**

1. Implementa la función heurística **PIEZAS DESCOLOCADAS** que se ha visto en clase de teoría ($h(n)=W(n)$) para una búsqueda de tipo A.
2. Implementa la función heurística **EUCLIDEAN**: Esta es una búsqueda de tipo A que utiliza la distancia Euclídea como función heurística; la expansión de los nodos se realiza siguiendo la función $f(n)=g(n)+D(n)$, donde $g(n)$ es el factor coste correspondiente a la profundidad del nodo n en el árbol de búsqueda y $h(n)=D(n)$ es la distancia Euclídea, es decir, la suma de las distancias en línea recta de cada ficha del puzzle desde su posición actual a la posición que ocupará en el objetivo (raíz cuadrada de la suma de los cuadrados de las distancias horizontal y vertical).
3. Implementa la función heurística **FILAS_COLUMNAS** ($h(n)=FilCol(n)$) para una búsqueda de tipo A. Esta heurística funciona del siguiente modo: para cada ficha del tablero, si la ficha no está en su fila destino correcta se suma 1 punto; si la ficha no está en su columna destino correcta se suma 1 punto. Por tanto, el mínimo valor de esta heurística para una ficha es 0 (cuando la ficha

Práctica 1: Búsqueda en un espacio de estados

está colocada correctamente en su posición objetivo), y el máximo valor es 2 (cuando ni la fila ni la columna de la posición de la ficha son iguales a valor de la fila y columna de la posición objetivo).

4. Implementa la función heurística **EUCLIDEAN + LC**
5. Implementa una heurística inventada por ti como una combinación de las anteriores. No tiene que ser A*. La idea es intentar proponer una heurística que creas que funciona bien.
6. Por cada heurística, se pide un estudio comparativo con el resto de algoritmos que incluya
 - Determinar si la heurística es admisible y consistente, justificando las respuestas. Para estudiar la admisibilidad, comprobar que la heurística vale 0 en el estado objetivo, que no devuelve valores negativos y comparar, en varios ejemplos, su valor con el coste de la solución óptima. Para estudiar la consistencia, utilizar la función `check_consistency` con diferentes estados iniciales. La ausencia de contraejemplos aporta evidencia experimental, pero no constituye por sí sola una demostración.
 - Mostrar resultados en tablas como la siguiente donde poder justificar las respuestas usando datos concretos.

	BFS	DFS (Back)	Voraz	ID	Manh	MD + LC	Euclid	IDA* Manh	Desc	FilCol	Euclid + LC	Propu esta
Nodes generated												
Nodes expanded												
Max nodes stored												
Solution cost												
Search depth												
Running time												

7. Un estudio detallado de análisis entre las dos mejores heurísticas que hayáis identificado haciendo uso del notebook “alg-compare-p3.ipynb”. Justificando las respuestas

Anexo 1. Cómo usar Jupyter Notebook o Google Colab

Opción 1: Usar Jupyter Notebook (recomendado si se trabaja en local)

1. Asegúrate de tener instalado Python y Jupyter Notebook. Puedes instalarlo con:

```
pip install notebook
```

2. Sitúate en la carpeta donde están los archivos de la práctica (search.py, puzzle-p1.ipynb, etc.).

3. Ejecuta el siguiente comando en terminal:

```
jupyter notebook
```

4. Se abrirá el navegador. Haz clic en el fichero .ipynb que quieras utilizar (por ejemplo, puzzle-p1.ipynb).

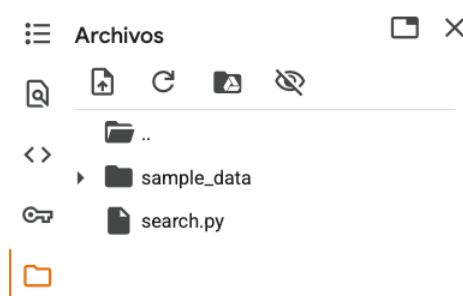
5. Ejecuta las celdas con Shift + Enter.

Nota: Asegúrate de que el archivo search.py esté en el mismo directorio para que las funciones se importen correctamente.

Opción 2: Usar Google Colab

1. Accede a <https://colab.research.google.com>.

2. Sube el archivo "search.py" desde el icono de carpeta > pestaña "Archivos" > "Subir". Debe quedar como en la imagen.



3. Abre el notebook deseado (por ejemplo, puzzle-p2.ipynb) con "Archivo > Subir notebook".

4. Añade, si no está, una celda al principio del notebook para importar el archivo search.py con:

```
from search import *
```

5. Ejecuta las celdas con Shift + Enter.

Nota: Si necesitas volver a subir search.py después de modificarlo, reinicia el entorno de ejecución (Entorno de ejecución > Reiniciar) y vuelve a ejecutarlo.